O que são OBJETOS?

Objetos são coleções **NÃO ordenadas** de valores, organizados em **pares de chave-valor**.

```
const obj = {
        chave : valor ,
        chave : valor ,
        chave : valor ,
};
```

Pares de chave-valor são inseridos entre chaves **{}**, usando a sintaxe **chave:valor** e separados por vírgula.

Object no MDN

a

Sintaxe do Objeto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
};
```

É possível agrupar dados em um objeto, por exemplo um **objeto cliente** com todos os dados de determinada cliente.

Sintaxe do Objeto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  endereco: {
    logradouro: 'Rua A',
    num: '125A',
    complemento: 'apto 24',
    //restante endereço
  },
  tags: ['premium', 'clube'],
};
```

Os valores em um objeto podem ser **qualquer tipo de dado JS**, inclusive **outros objetos**

Os dados podem estar organizados em **arrays dentro de objetos**

a

Sintaxe do Objeto

```
const clientes = [
  {
    nome: 'Joana A',
    cpf: '12345678987',
    email: 'j@j.com',
  },
  {
    nome: 'Bruno B',
    cpf: '23456789123',
    email: 'b@b.com',
  },
  {
    nome: 'Carlos M',
    cpf: '56789012345',
    email: 'c@c.com',
  },
];
```

É comum a organização de dados em **arrays de objetos**.

Cada objeto, independente da quantidade de **propriedades**, é **um único elemento** do array.

clientes[2] retorna o terceiro objeto do array.

a

Manipulando objetos - notação de ponto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  endereco: {
    logradouro: 'Rua A',
    num: '125A',
    complemento: 'apto 24',
  },
  tags: ['premium', 'clube'],
};

console.log(cliente.nome); //string
console.log(cliente.score); //número
console.log(cliente.endereco); //objeto
console.log(cliente.tags); //array
```

Propriedades podem ser acessadas com a **notação de ponto:**

objeto.chave

a

Manipulando objetos - notação de ponto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  endereco: {
    logradouro: 'Rua A',
    num: '125A',
    complemento: 'apto 24',
  },
  tags: ['premium', 'clube'],
};

console.log(cliente.nome); //string
console.log(cliente.score); //número
console.log(cliente.endereco); //objeto
console.log(cliente.tags); //array
```

Propriedades acessadas pela **chave** vão retornar o **valor** associado à chave.

a

Manipulando objetos - notação de ponto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  endereco: {
    logradouro: 'Rua A',
    num: '125A',
    complemento: 'apto 24',
  },
  tags: ['premium', 'clube'],
};

console.log(cliente.endereco.logradouro); //string
console.log(cliente.tags[0]); //string
```

É possível acessar objetos dentro de objetos usando a mesma **notação de ponto**.

Arrays podem ser acessados e manipulados a partir da chave.

Manipulando objetos - notação de ponto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
};

cliente.celular = '551199999999';
console.log(cliente);
cliente.celular = '551199999998';
console.log(cliente.celular);
```

Propriedades podem ser **adicionadas** a um objeto existente.

Se a propriedade não existe, **será criada**.
Se a propriedade já existe, **o valor será sobrescrito**.

a

Objetos - notação de colchete

```
const cliente = {
  'nome completo': 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
};

console.log(cliente['nome completo']);
```

Chaves com nomes complexos podem ser declaradas como strings e acessadas via **notação de colchete.**

Objetos - notação de colchete

```
const descrChave = 'nome';

const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
};

console.log(cliente[descrChave]); //Joana A
```

A notação de colchete também é usada quando a chave é recebida como um dado **variável**.

Métodos internos de Objetos

```
const cliente = {
  nome: 'Joana A',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
  saudarCliente: function() {
    console.log(`boas vindas, ${this.nome}`);
  }
};

cliente.saudarCliente();
//boas vindas, Joana A
```

Objetos podem receber chaves com o valor de **funções, que são executadas no contexto do objeto**.

Essas funções recebem o nome de **método.**

Métodos são acessados e executados usando a **notação de ponto**.

Métodos internos de Objetos

```
const cliente = {
  nome: "Joana A",
  score: 865,
  recorrente: true,
  tags: ["premium", "clube"],
  saudarCliente() {
    console.log(`boas vindas, ${this.nome}`);
  },
};
```

Métodos podem ser declarados com a **sintaxe mais curta**.

a

Métodos internos de Objetos

```
const cliente = {
  nome: "Joana A",
  score: 865,
  recorrente: true,
  tags: ["premium", "clube"],
  saudarCliente() {
    console.log(`boas vindas, ${this.nome}`);
  },
};

cliente.informarScore = function informarScore() {
  console.log(`seu score é ${this.score}`);
};

cliente.informarScore();
//seu score é 865
```

Métodos podem ser **adicionados** a um objeto já declarado.

this.propriedade dá contexto de execução para o método.

Métodos de Objeto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
};

console.log(Object.keys(cliente));
//[ 'nome', 'cpf', 'email', 'score', 'recorrente', 'tags' ]
```

O método **Object.keys(objeto)** retorna um array de **strings** contendo todas as **chaves** do objeto.

métodos de Object no MDN

a

Métodos de Objeto

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
};

console.log(Object.values(cliente));
//[ 'Joana A', '12345678987', 'j@j.com', 865, true, [ 'premium', 'clube' ]]
```

O método **Object.values(objeto)** retorna um array contendo todos os **valores** do objeto, sem convertê-los para string.

🚨 Ao usar **Object.keys()** e **Object.values()** lembre que objetos **não são coleções ordenadas.**

a

Métodos de Objeto

```
const cliente = {
  nome: 'Joana A',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
};

console.log(Object.entries(cliente));
// [
//   [ 'nome', 'Joana A' ],
//   [ 'score', 865 ],
//   [ 'recorrente', true ],
//   [ 'tags', [ 'premium', 'clube' ] ]
// ]
```

O método **Object.entries(objeto)** retorna um array bidimensional contendo os pares de chave e valor no formato **[chave, valor]**.

As chaves são convertidas para string.

a

Iteração de Objetos com FOR...IN

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
};

for (const info in cliente) {
  const texto = `chave ${info}, valor do tipo ${typeof cliente[info]}`;
  console.log(texto);
}
```

```
chave nome, valor do tipo string
chave cpf, valor do tipo string
chave email, valor do tipo string
chave score, valor do tipo number
chave recorrente, valor do tipo boolean
chave tags, valor do tipo object
```

a

Iteração de Objetos com FOR...IN

```
const cliente = {
  nome: 'Joana A',
  cpf: '12345678987',
  email: 'j@j.com',
  score: 865,
  recorrente: true,
  tags: ['premium', 'clube'],
};

for (const info in cliente) {
  const texto = `chave ${info}, valor do tipo ${typeof cliente[info]}`;
  console.log(texto);
}
```

O laço **for...in** itera todas as propriedades **enumeráveis** de um objeto, através de suas **chaves**.

for...in no MDN

a

Compartilhe um resumo de seus novos conhecimentos em suas redes sociais.
#aprendizadoalura

alura | Escola Programação